МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И
МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
бюджетное образовательное учреждение высшего образования
«Московский технический университет связи и информатики»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №1

«Проектирование и создание структуры базы данных»

Выполнил: студент группы БВТ2504
Воронин А.А.

Москва, 2026 г.

Оглавление

1. Постановка задачи.....3
2. Анализ предметной области и бизнес-правила.....3
3. ER-модель.....4
4. Нормализация.....4
5. Обоснование ключей.....4
6. Словарь данных.....5
7. Результаты выполнения.....5
8. Выводы.....5

1. Постановка задачи

Целью работы является проектирование и физическая реализация структуры базы данных маркетплейса в СУБД PostgreSQL на основе анализа предметной области.

База данных должна удовлетворять требованиям 3НФ, содержать не менее 10 сущностей, включать иерархические связи, связи M:N, а также ограничения целостности (PK, FK, UNIQUE, CHECK, NOT NULL, DEFAULT).

2. Анализ предметной области и бизнес-правила

Выделены 13 сущностей: users, sellers, categories, products, stock, carts, cart_items, orders, order_items, payments, reviews, addresses, deliveries.

Ключевые бизнес-правила:

1. Продавец - пользователь (1:1).

2. Товар принадлежит одному продавцу и одной категории.

3. Категории - иерархия (самоссылка parent_id).

4. Заказ ↔ товары - связь M:N через order_items.

5. ON DELETE RESTRICT для товаров в заказах, CASCADE для позиций заказа.

6. Цены - NUMERIC(12,2), CHECK (price > 0).

7. Склад (stock) — связь 1:1 с товаром. Количество не может быть отрицательным.

8. Отзыв (reviews) — один пользователь может оставить один отзыв на один товар (уникальная пара user_id, product_id). Оценка от 1 до 5.

9. Доставка (deliveries) — связь 1:1 с заказом, ссылается на адрес. Статус (enum)

3. ER-модель

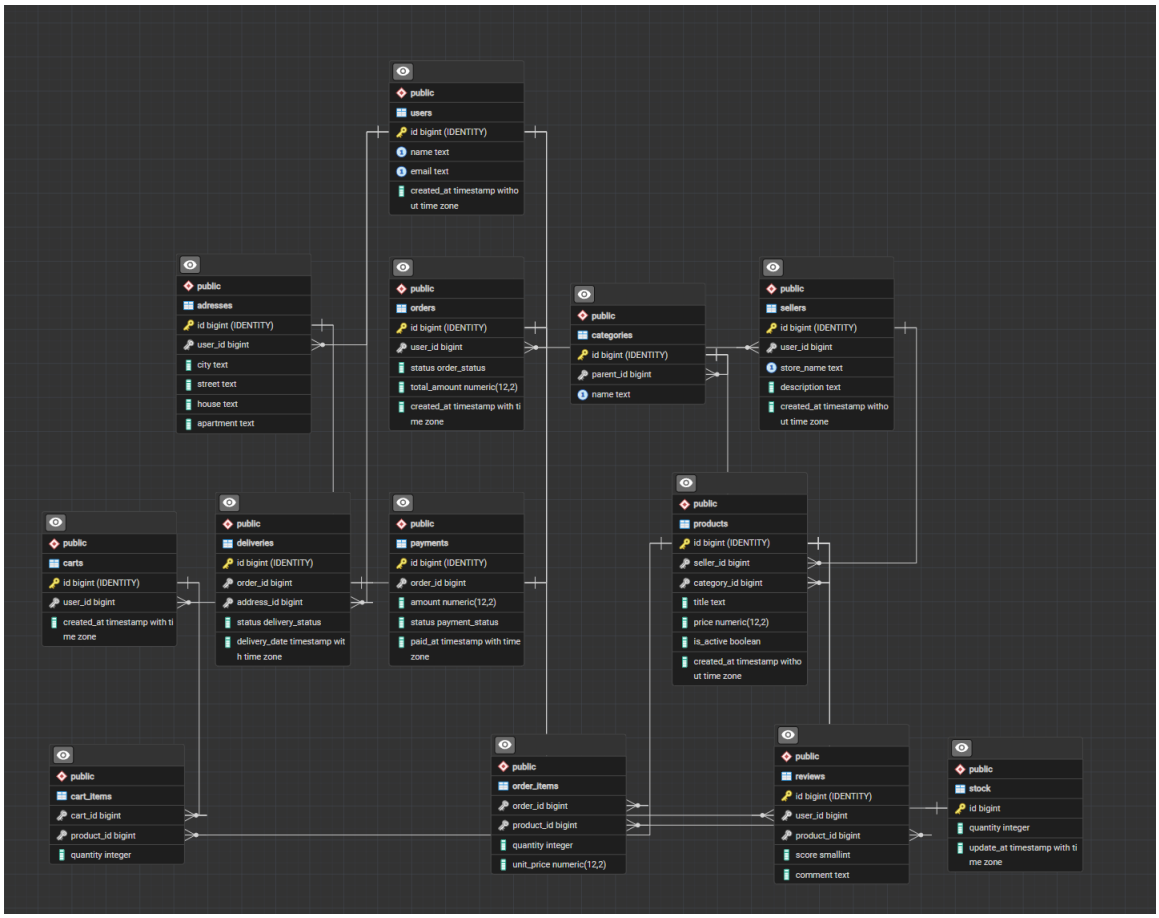


Схема включает в себя 13 сущностей, связи 1:N, M:N (через order_items, cart_items), самоссылку (categories.parent_id), связи 1:1 (users↔sellers, orders↔payments).

4. Нормализация

Схема в 3НФ. Пример аномалии до нормализации: хранение store_name и seller_phone в таблице products приводит к аномалиям обновления, вставки и удаления. Решение - вынос в отдельную таблицу sellers.

5. Обоснование ключей

Суррогатные ключи INTEGER GENERATED ALWAYS AS IDENTITY: стабильны, компактны, не зависят от бизнес-правил. Естественные уникальные атрибуты (email, shop_name) поддержаны через UNIQUE.

6. Словарь данных

Для каждой из 13 таблиц перечислены столбцы, типы (INTEGER, VARCHAR, NUMERIC, TIMESTAMPTZ, BOOLEAN), ограничения (PK, FK, NOT NULL, UNIQUE, CHECK, DEFAULT) и назначение.

7. Результаты выполнения

DDL-скрипт lab1voronin.sql выполнен без ошибок. Создано 13 таблиц, наложены все ограничения целостности. Скрипт воспроизводим: повторный запуск пересоздаёт схему с нуля удаляя прошлые таблицы, которые могли помешать с воспроизведением скрипта.

8. Выводы

Мне удалось спроектировать и физически реализовать структуры базы данных маркетплейса в СУБД PostgreSQL на основе анализа предметной области.

Приложение А. DDL-скрипт lab1.sql

```
drop table if exists users, sellers, categories, products, stock, carts,
cart_items, order_items, payments, reviews, orders, deliveries, addresses;
drop type if exists order_status, payment_status,delivery_status;

create type order_status as enum (
    'created', 'paid', 'shipped', 'delivered', 'cancelled'
);

create type payment_status as enum (
    'pending', 'completed', 'failed', 'refunded'
);

create type delivery_status as enum (
    'pending', 'in_transit', 'delivered', 'returned'
);

create table users (
  id bigint generated always as identity primary key,
  Name text not null unique,
  email text not null unique,
  created_at timestamp not null default now()
);

create table sellers (
    id bigint generated always as identity primary key,
user_id bigint references users(id),
    store_name text not null unique,
description text,
    created_at timestamp not null default now()
);

create table categories (
    id bigint generated always as identity primary key,
    parent_id bigint references categories(id) on delete restrict,
    name text not null unique
);

create table products (
    id bigint generated always as identity primary key ,
    seller_id bigint not null references sellers(id) on delete restrict,
    category_id bigint not null references categories(id),
    title text not null ,
    price numeric(12,2) not null check (price > 0),
    is_active boolean not null default true,
```

```
    created_at timestamp not null default now()
);

create table stock(
id bigint primary key references products(id) on delete cascade,
quantity integer not null default 0 check (quantity >= 0),
update_at timestamptz not null default now()
);

create table carts (
id bigint generated always as identity primary key,
user_id bigint not null references users(id) on delete cascade,
created_at timestamptz not null default now()
);

create table cart_items (
cart_id bigint not null references carts(id) on delete cascade,
product_id bigint not null references products(id) on delete cascade,
quantity integer not null check (quantity > 0)
);

create table orders (
id bigint generated always as identity primary key,
user_id bigint not null references users(id) on delete restrict,
status order_status not null default 'created',
total_amount numeric(12,2) not null default 0 check (total_amount >= 0),
created_at timestamptz not null default now()
);

create table order_items (
order_id bigint not null references orders(id) on delete cascade,
product_id bigint not null references products(id) on delete restrict,
quantity integer not null check (quantity > 0),
unit_price numeric(12,2) not null check (unit_price > 0)
);

create table payments (
id bigint generated always as identity primary key,
order_id bigint not null unique references orders(id) on delete cascade,
amount numeric(12,2) not null check (amount > 0),
status payment_status not null default  'pending' ,
paid_at timestamptz
);

create table reviews (
```

```
id bigint generated always as identity primary key,
user_id bigint not null references users(id) on delete cascade,
product_id bigint not null references products(id) on delete cascade,
score smallint not null check (score between 1 and 5),
comment text,
unique (user_id , product_id)
);

create table adresses (
id bigint generated always as identity primary key,
user_id bigint not null references users(id) on delete cascade,
city text not null,
street text not null,
house text not null,
apartment text not null
);

create table deliveries (
id bigint generated always as identity primary key,
order_id bigint not null unique references orders(id) on delete cascade,
address_id bigint not null references adresses(id) on delete restrict,
status delivery_status not null default 'pending',
delivery_date timestamptz
);
```